Taller 2 – Agente DQN para Atari Breakout

Erwin Arnoldo Daza Rendón

Nicolás Rozo Hincapié

Fabián Eduardo Ruiz Sánchez

Andrés Enrique Pachón Bustos

Facultad de Ingeniería, Universidad de La Sabana

Simulación y Aprendizaje por Refuerzo

Dr. (c) Emilio Muñoz Perez

04 de octubre de 2026

TABLA DE CONTENIDO

INTRODUCCIÓN 3
Juego seleccionado 3
2. Características del ambiente 3
Espacio de observaciones 3
Espacio de acciones 4
Sistema de recompensas 5
Condiciones de terminación 5
3. Ajustes realizados en el código 5
4. Ajuste de hiperparámetros 6
5. Flujo lógico del entrenamiento 7
6. Explicación de la red neuronal 9
7. Resultados del entrenamiento 10
8. Reflexión sobre los resultados 14
9. Reflexión sobre los retos y lo que más costó 15
10. Cómo ejecutar 16
Referencias 16

INTRODUCCIÓN

Actividad 2 de Aprendizaje por Refuerzo. En este trabajo entrenamos un agente Deep Q-Network (DQN) para que aprenda a jugar Breakout de Atari usando el entorno ALE/Breakout-v5 de Gymnasium. Tomamos como base el proyecto rl_games visto en clase, que originalmente estaba pensado para LunarLander y CartPole, y lo adaptamos a un juego de Atari siguiendo el artículo de Mnih et al. (2015), "Human-level control through deep reinforcement learning", junto con la documentación de ALE y de los wrappers de Gymnasium.

Juego seleccionado

Escogimos Breakout porque no lo habíamos trabajado en clase y porque es uno de los juegos con los que se presentó DQN en el artículo, así que teníamos una referencia clara para comparar. Además, nos pareció un buen reto pasar de entornos donde la observación es un vector pequeño de números, como en LunarLander, a uno donde el agente solo ve la imagen de la pantalla.

En Breakout el jugador mueve una paleta en la parte de abajo de la pantalla y debe hacer rebotar una bola para romper un muro de ladrillos que está arriba. Cada vez que la bola pasa por debajo de la paleta se pierde una vida, y el jugador empieza con cinco.

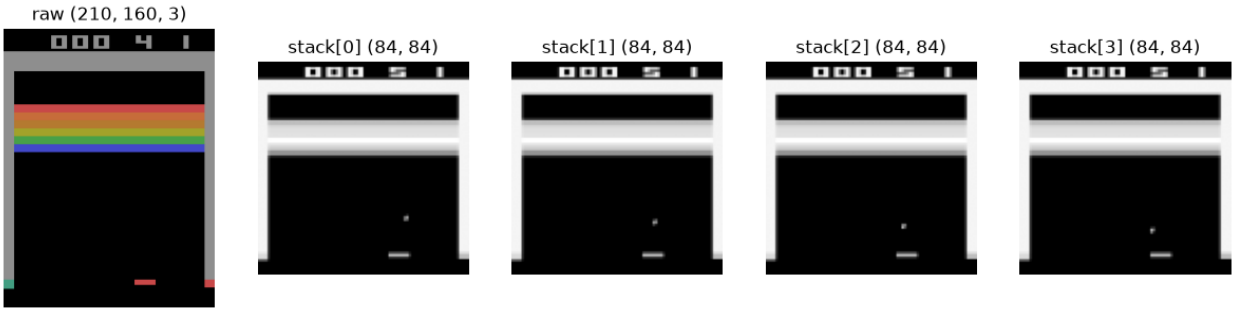
2. Características del ambiente.

Espacio de observaciones

El entorno original entrega una imagen RGB de 210x160 píxeles con valores entre 0 y 255. Trabajar con esa imagen directamente es muy pesado, entonces hicimos el mismo preprocesamiento que propone el artículo: cada imagen se pasa a escala de grises, se reduce a 84x84 y se apilan las últimas 4 imágenes. Con eso la observación que recibe la red queda de forma (4, 84, 84) y tipo uint8. Se apilan cuatro imágenes porque con una sola no se sabe para dónde va la bola ni qué tan rápido se mueve. Al tener la historia reciente del movimiento de la bola, el agente puede saber hacia dónde se dirige y así tomar una mejor decisión (ver Figura 1 y Figura 2).

Figura 1

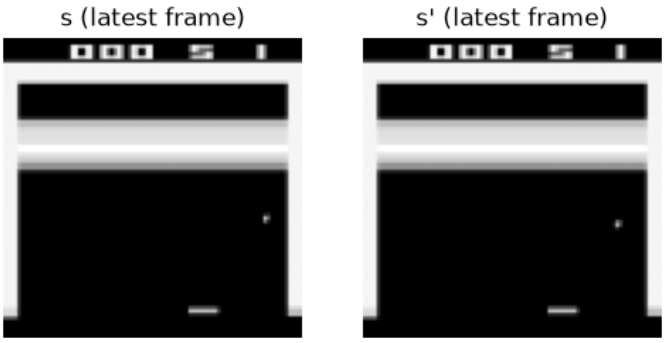
Cuadro original del juego a color de 210x160 y la pila de 4 imágenes grises de 84x84 que recibe la red



Nota. Tomado del notebook breakout_dqn.ipynb del repositorio rl_games_sb (Muñoz, 2026).

Figura 2

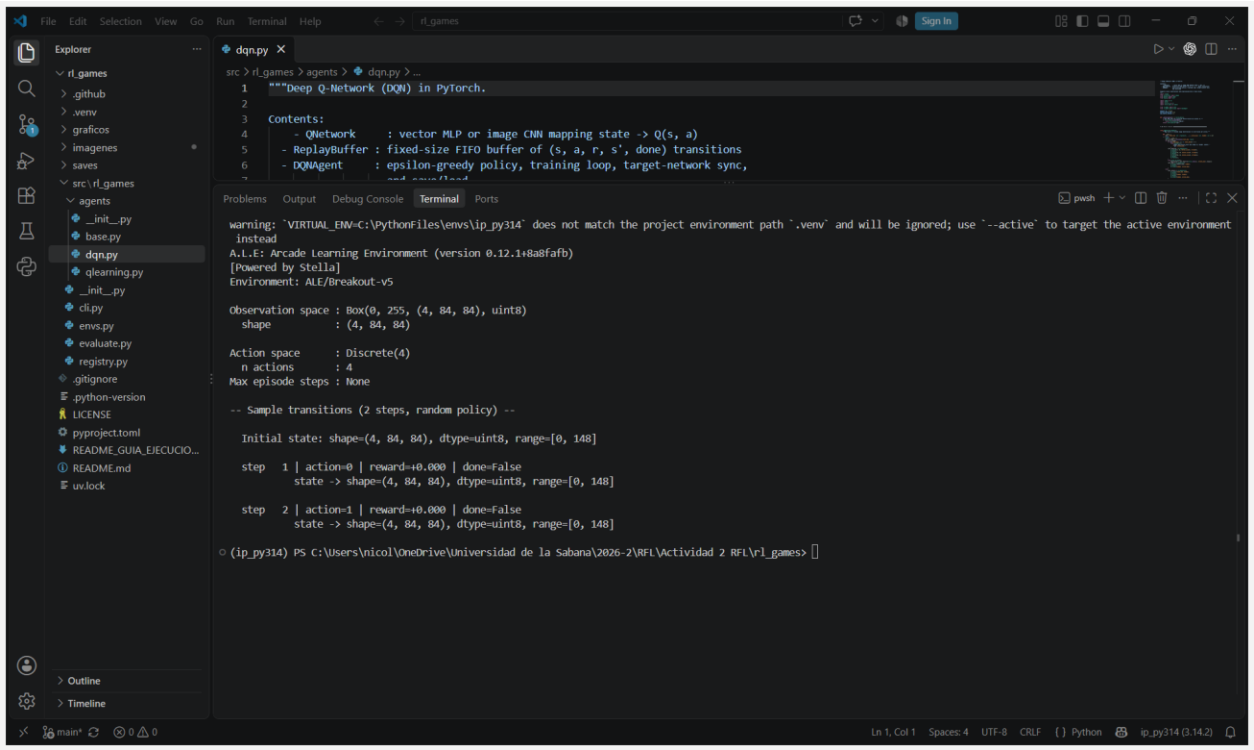
Última imagen del estado actual (s) y del siguiente estado (s'). Entre una y otra la bola cambia de posición



Nota. Tomado del notebook breakout_dqn.ipynb del repositorio rl_games_sb (Muñoz, 2026).

Figura 3

Ejecución de rlgames inspect en la terminal de VS Code. Se ve el espacio de observaciones Box(0, 255, (4, 84, 84), uint8) y el espacio de acciones Discrete(4).



Espacio de acciones

El espacio de acciones es discreto con 4 opciones: 0 es NOOP (no hacer nada), 1 es FIRE (lanzar la bola), 2 es RIGHT (mover la paleta a la derecha) y 3 es LEFT (moverla a la izquierda). ALE también permite

usar las 18 acciones del control completo de Atari, pero para Breakout no tiene sentido porque la mayoría harían lo mismo.

Sistema de recompensas

El agente recibe puntos cada vez que rompe un ladrillo, y el valor depende del color de la fila. Las dos filas de abajo dan 1 punto, las dos del medio dan 4 y las dos de arriba dan 7. En el resto de los pasos la recompensa es 0 y perder una vida no da recompensa negativa. Esto hace que la señal sea escasa al principio, porque un agente que se mueve al azar casi nunca le pega a la bola.

Condiciones de terminación

El episodio termina cuando el jugador pierde las cinco vidas. También existe un límite de 108000 frames por episodio, que equivale a unos 30 minutos de juego; si se llega a ese límite el episodio se corta (truncated). En nuestro proyecto un episodio es la partida completa, es decir, perder una vida no termina el episodio.

3. Ajustes realizados en el código

El proyecto base traía la estructura del agente, el buffer y la línea de comandos, pero la red neuronal y el paso de aprendizaje estaban como ejercicio sin resolver, y el entorno por defecto era LunarLander. Estos fueron los cambios principales. El código desarrollado y su historial de commits se pueden consultar en https://github.com/erwindare/rl_games.

En pyproject.toml cambiamos gymnasium[box2d] por gymnasium[atari] para tener los juegos de Atari, y agregamos opencv-python-headless para el redimensionamiento de las imágenes, pygame para poder ver el juego en una ventana y matplotlib para la gráfica de aprendizaje.

En envs.py dejamos ALE/Breakout-v5 como único entorno y armamos la cadena de preprocesamiento con los wrappers de Gymnasium:

```
env = gym.make(env_id, render_mode=render_mode, frameskip=1, repeat_action_probability=0.0)
env = gym.wrappers.AtariPreprocessing(env, frame_skip=4, screen_size=84, grayscale_obs=True,
scale_obs=False)
env = gym.wrappers.FrameStackObservation(env, stack_size=4)
return _FireOnResetAndLifeLoss(env)
```

El entorno se crea con frameskip=1 porque AtariPreprocessing es el que se encarga de saltar los frames y no deja usarlo si el entorno ya trae su propio salto. También se dejó repeat_action_probability=0.0, que por defecto en la versión 5 es 0.25 y hace que a veces el juego repita la acción anterior en vez de la elegida. Además, se creó el wrapper _FireOnResetAndLifeLoss, que presiona FIRE al iniciar cada partida y también 3 pasos después de perder una vida. Sin esto la bola nunca sale y el agente se puede quedar quieto toda la partida sin aprender nada.

En agents/dqn.py se completó la red QNetwork con una parte convolucional para imágenes, se implementó la selección de acciones épsilon-greedy y el paso de aprendizaje _learn. El ReplayBuffer se modificó para que con imágenes guarde todo en arreglos de uint8 y del siguiente estado solo guardamos

la imagen nueva, ya que las otras tres ya están en el estado actual; con eso se ahorra bastante memoria. También se agregó la detección automática de GPU, se cambió la pérdida de MSE a Huber (SmoothL1Loss), se agregó recorte de gradientes y, en el segundo commit, una función que al final del entrenamiento guarda la curva de aprendizaje en la carpeta graficos.

Por último, en cli.py y registry.py se dejó solo el agente DQN y el entorno de Breakout, y se cambió la forma de imprimir las observaciones para que muestre la forma y el rango de la imagen en vez de imprimir miles de números.

4. Ajuste de hiperparámetros

La idea fue partir de los valores del artículo (Extended Data Table 1) y cambiar solo lo que no era posible replicar con los recursos que teníamos. La siguiente tabla compara ambos:

Tabla 1

Hiperparámetros del artículo (Extended Data Table 1) frente a los de nuestro proyecto

Parámetro	Artículo (Mnih et al., 2015)	Nuestro proyecto
Optimizador	RMSProp	Adam
Tasa de aprendizaje	0.00025	0.00025
Tamaño del lote	32	32
Factor de descuento gamma	0.99	0.99
Frames apilados	4	4
Repetición de cada acción	4 frames	4 frames
Acciones nulas aleatorias al inicio	hasta 30	hasta 30
Memoria de repetición	1 millón de transiciones	10 mil transiciones
Actualización de la red objetivo	cada 10 mil actualizaciones	cada 10 episodios
Exploración épsilon	de 1.0 a 0.1 en el primer millón de frames	de 1.0 a 0.01, multiplicando por 0.995 cada episodio
Inicio del aprendizaje	después de 50 mil transiciones	después de 32 transiciones
Frecuencia de actualización	cada 4 acciones	cada acción
Neuronas de la capa densa	512	128
Recorte de recompensas	sí, entre -1 y 1	no
Duración del entrenamiento	50 millones de frames	1000 episodios

La tasa de aprendizaje, el lote, el gamma, el apilado de 4 imágenes y el salto de 4 frames se dejaron igual que en el artículo. Se usó Adam en vez de RMSProp porque es el optimizador que ya traía el proyecto y suele ser más fácil de ajustar. La pérdida de Huber cumple un papel parecido al recorte del error que hacen en el artículo, porque evita que errores muy grandes produzcan saltos fuertes en los pesos.

El cambio más grande fue la memoria de repetición. Cada estado guardado ocupa unos 35 KB, así que 10 mil transiciones son cerca de 350 MB de RAM, mientras que el millón del artículo serían unos 35 GB, mucho más de lo que tienen nuestros computadores. Por la misma razón de recursos se pasó de medir el entrenamiento en frames a medirlo en episodios. La épsilon baja un poco al final de cada episodio y,

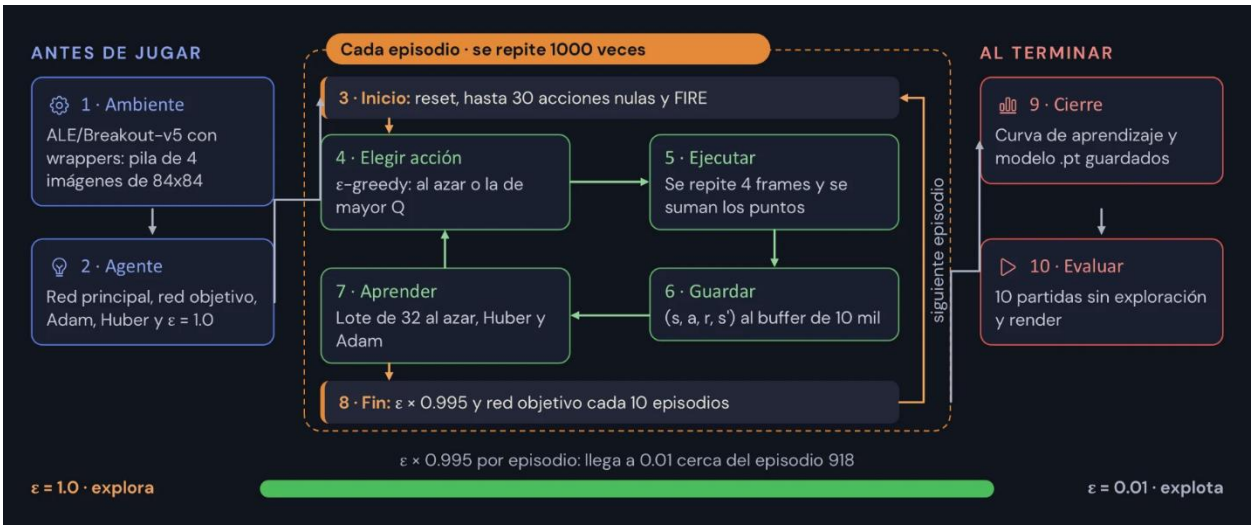
con un factor de 0.995, llega a su mínimo de 0.01 alrededor del episodio 918, o sea que el agente explora bastante en la primera mitad y casi nada en la última parte. La red objetivo se copia cada 10 episodios, que es la forma en que el proyecto ya manejaba esa sincronización.

5. Flujo lógico del entrenamiento

A continuación, se describe lo que pasa cuando se ejecuta rlgames train dqn --episodes 1000 (ver Figura 4).

Figura 4

Diagrama del flujo lógico del entrenamiento, del Paso 1 al Paso 10



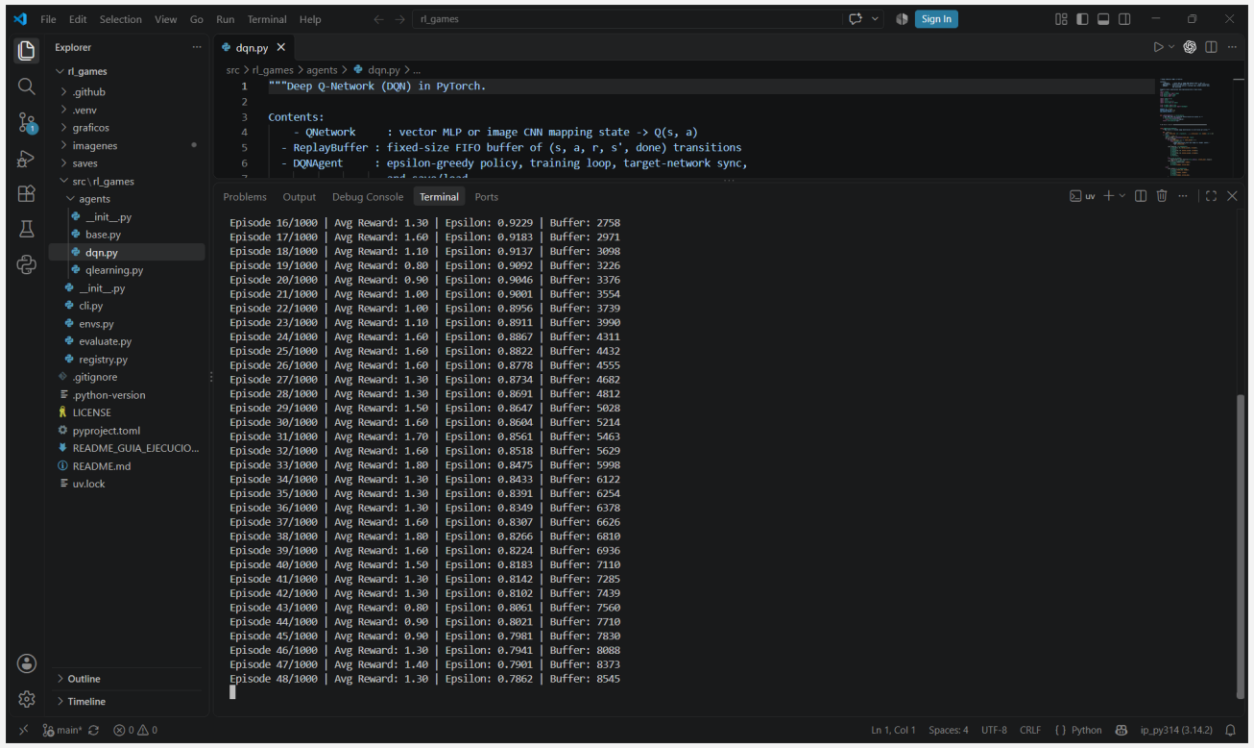
Nota. Elaboración propia.

- **Paso 1. Creación del ambiente**. Se registra ALE en Gymnasium y se crea ALE/Breakout-v5 con los wrappers descritos antes. Desde aquí el agente ya no ve la imagen a color de 210x160, sino la pila de 4 imágenes grises de 84x84.
- **Paso 2. Creación del agente**. Se crean dos redes con la misma arquitectura: la red principal, que es la que aprende, y la red objetivo, que arranca con una copia de los pesos de la principal. También se crea el optimizador Adam, la pérdida de Huber y un buffer vacío. La épsilon empieza en 1.0, es decir, al comienzo todas las acciones son aleatorias. Si hay GPU disponible las redes se pasan a la tarjeta gráfica.
- **Paso 3. Inicio de cada episodio.** Se reinicia el juego. AtariPreprocessing hace entre 0 y 30 acciones nulas al azar para que no todas las partidas empiecen exactamente iguales, y luego el wrapper propio presiona FIRE para lanzar la bola. Esta es una particularidad de Breakout: si nadie presiona FIRE el juego no arranca.
- **Paso 4. Selección de la** acción. Con probabilidad épsilon el agente escoge una de las 4 acciones al azar y, en caso contrario, le pasa la pila de imágenes a la red principal y escoge la acción con el valor Q más alto.

- **Paso 5. Ejecución de la acción.** La acción se repite durante 4 frames y se suman las recompensas de esos frames. Para la imagen se toma el máximo píxel a píxel entre los dos últimos frames, porque en Atari algunos objetos parpadean y solo aparecen en frames pares o impares. Si en ese paso se perdió una vida, el wrapper cuenta 3 pasos y vuelve a presionar FIRE para sacar una bola nueva.
- **Paso 6. Almacenamiento de la experiencia.** La transición (estado, acción, recompensa, siguiente estado, terminado) se guarda en el buffer. Cuando el buffer llega a 10 mil transiciones, las nuevas van reemplazando a las más viejas.
- **Paso 7. Aprendizaje.** En cada paso, apenas el buffer tiene al menos 32 transiciones, se toma un lote de 32 al azar. Tomarlas al azar rompe la relación que hay entre imágenes seguidas del mismo juego. Para cada transición se calcula el valor objetivo con la red objetivo:
  ```
  objetivo = recompensa + 0.99 * max Q_objetivo(siguiente estado) * (1 - terminado)
  ```
  Luego se compara con el valor Q que predice la red principal para la acción que realmente se tomó, se calcula la pérdida de Huber, se hace la retropropagación, se recortan los gradientes a una norma máxima de 10 y se actualizan los pesos con Adam.
- **Paso 8. Fin del episodio.** Cuando se pierden las cinco vidas o se alcanza el límite de frames, el episodio termina. Se multiplica épsilon por 0.995, se guarda la recompensa total del episodio y, si el número del episodio es múltiplo de 10, se copian los pesos de la red principal a la red objetivo. En la terminal se imprime una línea con el promedio de recompensa de los últimos 10 episodios, la épsilon actual y el tamaño del buffer.
- **Paso 9. Cierre del entrenamiento**. Al terminar los 1000 episodios se dibuja la curva de aprendizaje con la recompensa de cada episodio y su promedio móvil, se guarda en graficos/, y el modelo se guarda en saves/dqn_ALE_Breakout-v5.pt. Si se vuelve a correr el comando, el entrenamiento continúa desde ese archivo.
- **Paso 10. Evaluación**. Con rlgames load dqn --eval el agente juega 10 partidas sin exploración, siempre escogiendo la mejor acción según la red, y se calcula la recompensa promedio. Con rlgames render dqn se abre una ventana para verlo jugar.

Figura 5

Primeros episodios del entrenamiento. Se ve cómo baja la épsilon y cómo se va llenando el buffer.



6. Explicación de la red neuronal

La red recibe la pila de 4 imágenes de 84x84 y entrega un valor Q por cada una de las 4 acciones, todo en una sola pasada. Esa es la misma idea del artículo: en vez de meter la acción como entrada y correr la red una vez por acción, la red da todos los valores de una vez.

```
self.features = nn.Sequential(
    nn.Conv2d(4, 32, kernel_size=8, stride=4), nn.ReLU(),
    nn.Conv2d(32, 64, kernel_size=4, stride=2), nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, stride=1), nn.ReLU(),
    nn.Flatten(),
)
self.head = nn.Sequential(nn.Linear(3136, 128), nn.ReLU(), nn.Linear(128, 4))
```

Antes de entrar a la red los píxeles se dividen entre 255 para dejarlos entre 0 y 1. La primera capa convolucional[1] tiene 32 filtros de 8x8 con paso 4 y deja una salida de 32 mapas de 20x20; esta capa detecta cosas grandes y simples como bordes y la posición de la paleta. La segunda tiene 64 filtros de 4x4 con paso 2 y deja 64 mapas de 9x9. La tercera tiene 64 filtros de 3x3 con paso 1 y deja 64 mapas de 7x7, donde

[1] Una red neuronal convolucional (CNN) es un tipo de red pensada para procesar imágenes. En lugar de conectar cada píxel con todas las neuronas, aplica filtros pequeños que recorren la imagen y detectan patrones locales, como bordes o formas, y al apilar varias capas combina esos patrones en características más complejas (LeCun et al., 1998).

ya se combinan cosas como la posición de la bola respecto a la paleta y su movimiento entre las 4 imágenes. Después de cada capa se usa ReLU.

Esos 64 mapas de 7x7 se aplanan en un vector de 3136 valores, que pasa por una capa densa de 128 neuronas con ReLU y luego por la capa de salida de 4 neuronas, una por acción, sin función de activación porque los valores Q pueden ser cualquier número. En total la red tiene 480036 parámetros. Las tres capas convolucionales son iguales a las del artículo; la diferencia está en la capa densa, que allá tiene 512 neuronas, lo que da cerca de 1.7 millones de parámetros. Usar 128 hace la red más liviana y rápida de entrenar, aunque con menos capacidad.

7. Resultados del entrenamiento

En el primer entrenamiento de 1000 episodios el computador y tardó cerca de 3 horas. Después lo repetimos desde cero en un segundo computador con procesador AMD Ryzen 9 8940HX, 15 GB de RAM y tarjeta gráfica NVIDIA GeForce RTX 5060 Laptop, ejecutando todo desde la terminal de Visual Studio Code. En este equipo los 1000 episodios tardaron 38 minutos y 31 segundos.

Figura 6

Alrededor del episodio 500 el promedio de los últimos 10 episodios ya está cerca de 9 puntos y la épsilon bajó a 0.08.

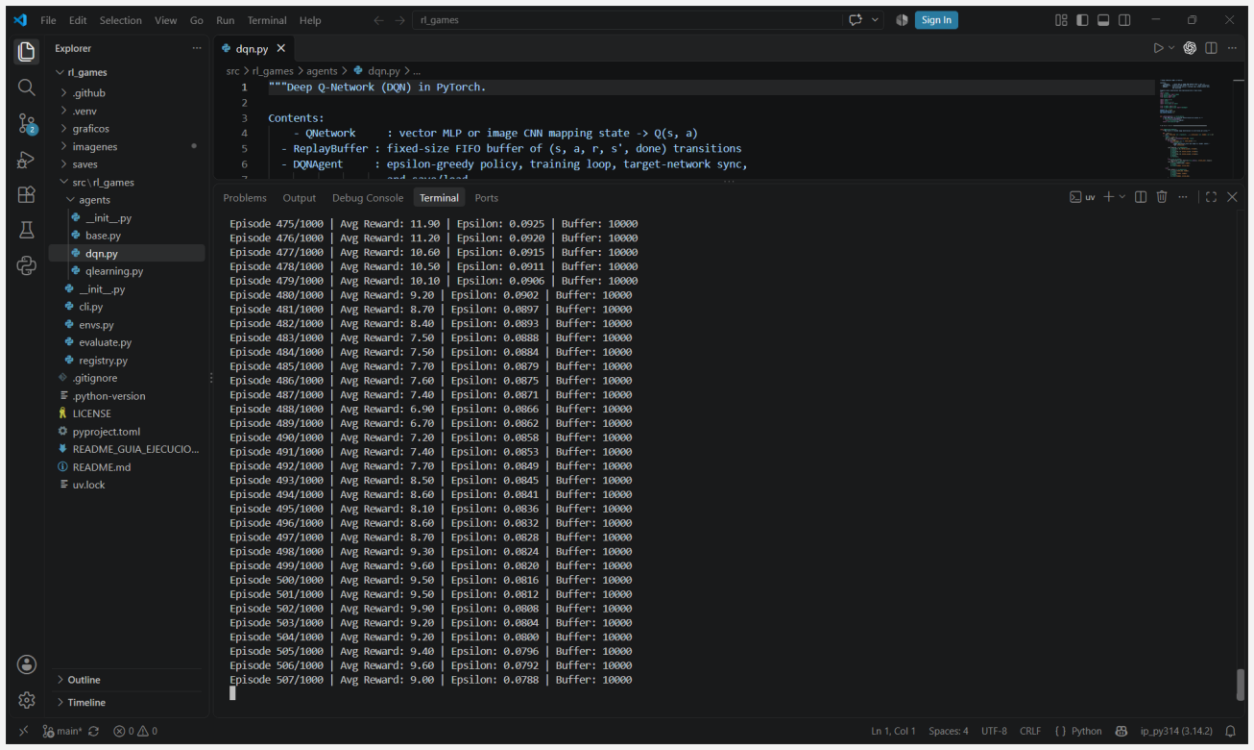
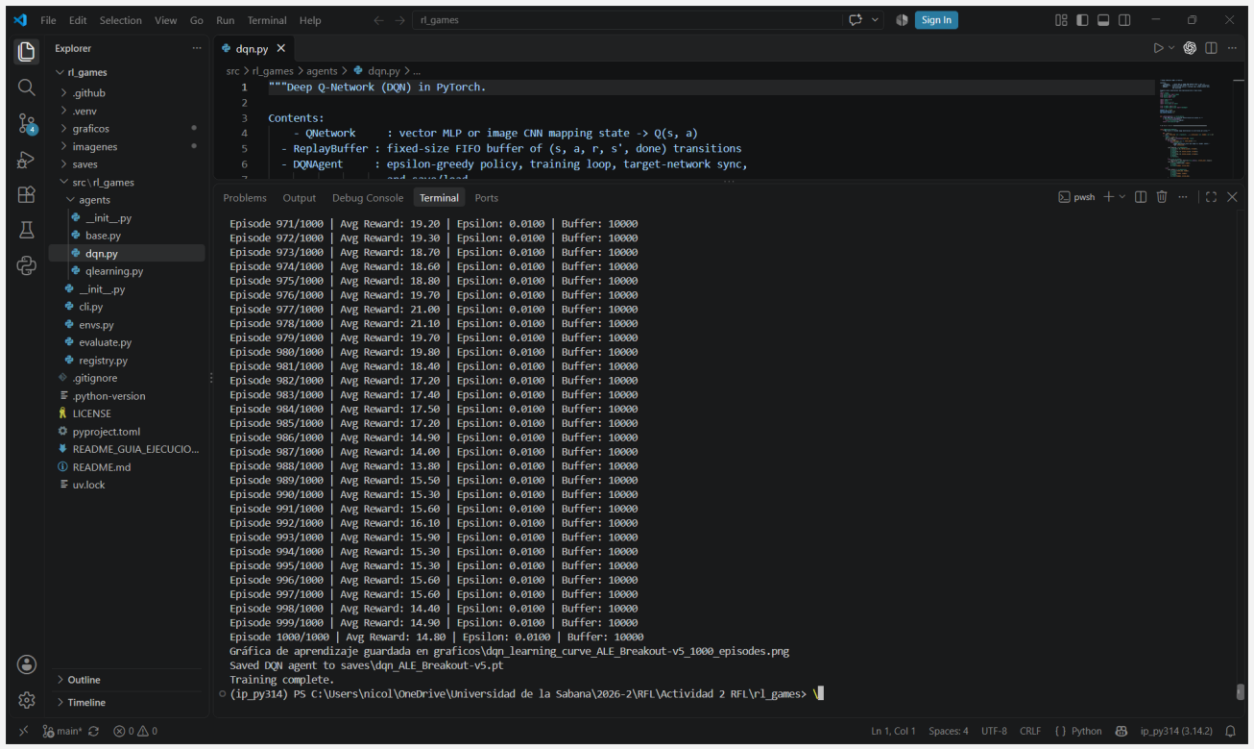


Figura 7

Final del entrenamiento: la épsilon está en su mínimo de 0.01, se guarda la gráfica y el modelo.



La tabla muestra la recompensa promedio cada 100 episodios, calculada a partir del registro que quedó en la terminal.

Tabla 2

Recompensa promedio, mejor episodio y épsilon cada 100 episodios

Episodios	Recompensa promedio	Mejor episodio	Épsilon al final del bloque
1 a 100	1.44	6	0.606
101 a 200	1.71	20	0.367
201 a 300	5.08	21	0.222
301 a 400	10.06	28	0.135
401 a 500	9.44	24	0.082
501 a 600	10.49	26	0.049
601 a 700	13.87	32	0.030
701 a 800	16.91	38	0.018
801 a 900	16.69	32	0.011
901 a 1000	16.42	47	0.010

El mejor episodio fue el 920, con 47 puntos, y el mejor promedio de 10 episodios seguidos fue de 23.1, en el episodio 804. En los últimos 500 episodios no hubo ninguna partida con 0 puntos.

Figura 8

Curva de aprendizaje del entrenamiento hecho en el segundo computador. En azul la recompensa de cada episodio y en naranja el promedio móvil de 10 episodios.

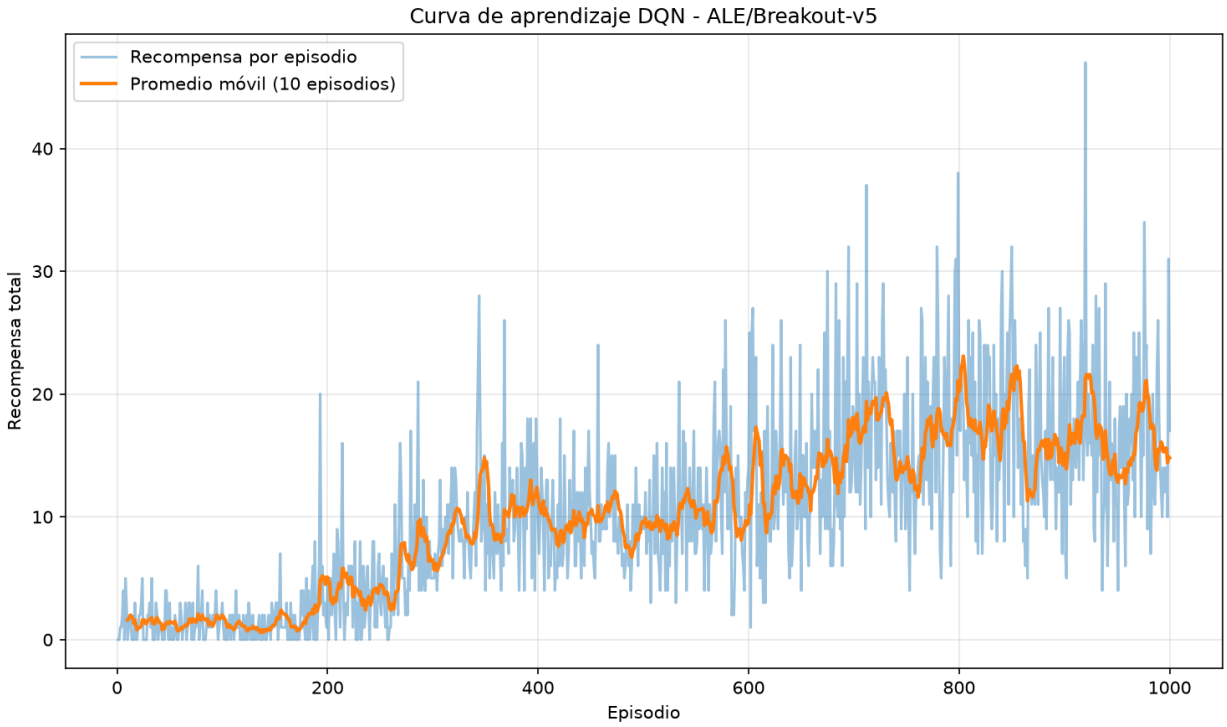
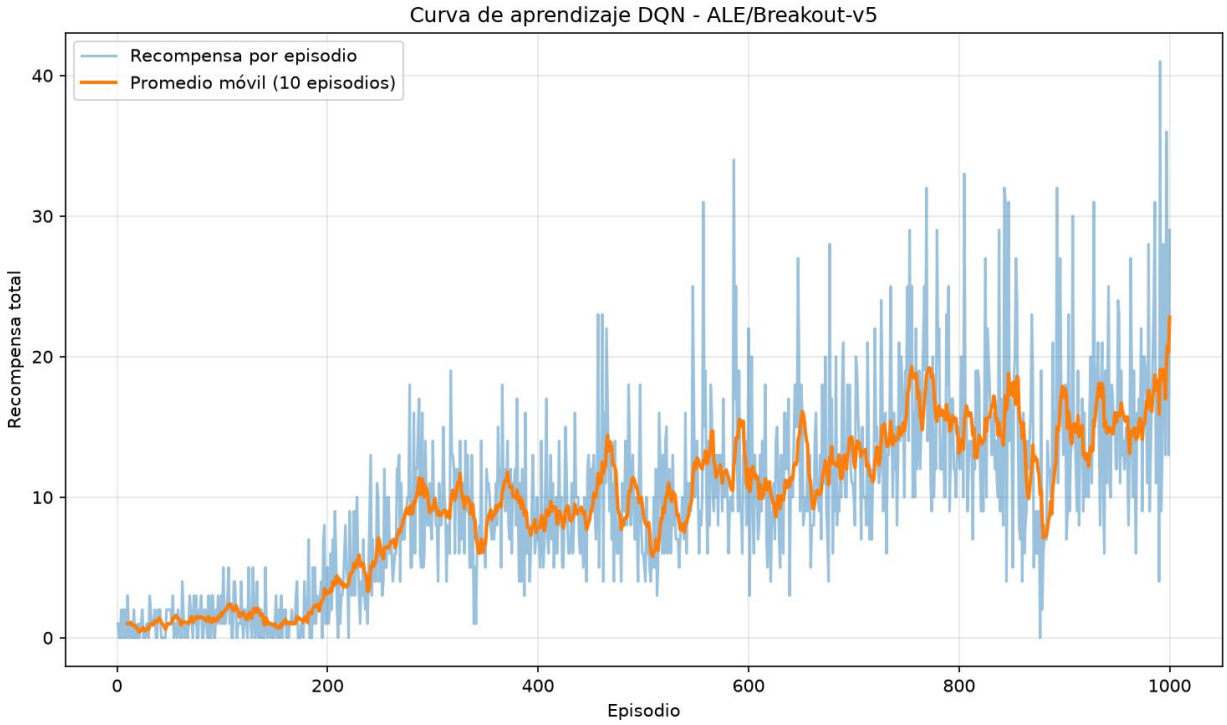


Figura 9

Curva de aprendizaje del primer entrenamiento, el de 3 horas, que está guardada en la carpeta graficos.



Las dos curvas se parecen mucho. En ambas el agente casi no aprende en los primeros 150 a 200 episodios, luego hay una subida fuerte hasta los 10 puntos entre los episodios 200 y 350, y después sigue subiendo más despacio hasta quedar entre 15 y 20 puntos de promedio. En el primer entrenamiento el mejor episodio fue de unos 41 puntos y en el segundo de 47.

Para la evaluación final el agente jugó 10 partidas sin exploración. Obtuvo 25, 24, 25, 24, 18, 18, 25, 25, 24 y 18 puntos, con un promedio de 22.6 y una desviación estándar de 3.04.

Figura 10

Resultado de rlgames load dqn --eval en la terminal de VS Code.

```
Using GPU: NVIDIA GeForce RTX 5060 Laptop GPU
DQN agent for ALE/Breakout-v5
  Episodes trained  : 1000
  Network params    : 480,036
  Epsilon           : 0.0100
  LR / Gamma        : 0.00025 / 0.99
  Batch size        : 32
  Target update     : every 10 episodes
  Device            : cuda:0

Evaluating (10 episodes) ...
Evaluation episode 1/10 | Reward: 25.00
Evaluation episode 2/10 | Reward: 24.00
Evaluation episode 3/10 | Reward: 25.00
Evaluation episode 4/10 | Reward: 24.00
Evaluation episode 5/10 | Reward: 18.00
Evaluation episode 6/10 | Reward: 18.00
Evaluation episode 7/10 | Reward: 25.00
Evaluation episode 8/10 | Reward: 25.00
Evaluation episode 9/10 | Reward: 24.00
Evaluation episode 10/10 | Reward: 18.00
  Mean reward: 22.60 +/- 3.04
(ip_py314) PS C:\Users\nicol\OneDrive\Universidad de la Sabana\2026-2\RFL\Actividad 2 RFL\rl_games>
```

Figura 11

Ventana de rlgames render dqn con el agente entrenado jugando. En ese momento llevaba 8 puntos y le quedaban 3 vidas.



8. Reflexión sobre los resultados

El agente sí aprendió a jugar. Pasó de un promedio de 1.4 puntos en los primeros 100 episodios, que es básicamente jugar al azar, a unos 16 puntos en los últimos 300 episodios de entrenamiento y 22.6 en la evaluación. Que la evaluación dé más alto que el entrenamiento tiene sentido, porque en la evaluación no hay acciones aleatorias y el agente siempre hace lo que cree que es mejor.

Para ponerlo en contexto, en la tabla de resultados del artículo un jugador al azar saca 1.7 puntos en Breakout, el probador humano 31.8 y el DQN de ellos 401.2. Nuestro agente está muy por encima del azar, pero todavía por debajo del humano, y muy lejos del resultado del artículo. Con 20 a 25 puntos por

partida ya logra devolver la bola varias veces, pero no alcanza a llegar a la estrategia que describe el artículo de abrir un túnel por un lado del muro para que la bola rompa los ladrillos de arriba sola.

También nos llamó la atención que entre los episodios 700 y 1000 el promedio casi no cambió, quedándose alrededor de 16 a 17 puntos, aunque la épsilon ya estaba muy baja. Creemos que parte de la razón es que el buffer de 10 mil transiciones solo alcanza para unas pocas decenas de episodios, entonces el agente aprende de experiencias muy recientes y parecidas entre sí. Además, la curva tiene mucha variación de un episodio a otro, con una desviación de casi 7 puntos en los últimos 100, lo que muestra que la política todavía no es estable.

En la evaluación los puntajes se repiten mucho, solo aparecen 18, 24 y 25. Como el agente juega sin aleatoriedad y la única variación al inicio son las acciones nulas, termina repitiendo casi las mismas jugadas. Esto quiere decir que el agente aprendió unos pocos patrones que le funcionan, pero no una forma general de jugar.

Finalmente, comparando con el artículo hay diferencias que seguramente afectan el resultado. Allá el episodio de entrenamiento se marcaba como terminado cada vez que se perdía una vida, lo que le da al agente una señal clara de que dejar caer la bola es malo. En nuestro caso perder una vida no da ninguna señal, simplemente se dejan de recibir puntos. Tampoco se recortan las recompensas, así que un ladrillo rojo de 7 puntos pesa mucho más que uno azul de 1, lo que puede hacer los valores Q más inestables.

9. Reflexión sobre los retos y lo que más costó

Lo que más nos costó fue la capacidad de cómputo. El artículo entrena cada juego durante 50 millones de frames, que según los autores equivale a unos 38 días de experiencia de juego. Eso era imposible para nosotros, así que se ajustó el número de episodios a 1000 en lugar de lo que sugiere el artículo. Aun así, con esos 1000 episodios el entrenamiento tardó cerca de 3 horas. Los resultados del modelo mejoraron bastante, pero no al punto de considerarlo perfecto, y para mejorarlo se necesitaría una capacidad de cómputo mucho más grande que permita más episodios, una memoria de repetición más grande y una red con más neuronas.

La memoria RAM fue otro límite. Guardar un millón de transiciones como en el artículo necesitaría unos 35 GB, por eso el buffer quedó en 10 mil. Fue necesario cambiar la forma en que el buffer guarda las imágenes para que el entrenamiento no se quedara sin memoria.

Cuando lo corrimos en el segundo computador, que tiene tarjeta gráfica NVIDIA, al principio PyTorch no la reconocía, porque en Windows uv sync instala la versión de PyTorch que solo usa el procesador. Tuvimos que instalar la versión con CUDA dentro del entorno virtual del proyecto y ejecutar los comandos con uv run --no-sync para que no se volviera a instalar la otra versión. Con la GPU el entrenamiento bajó de unas 3 horas a menos de 40 minutos, lo que deja ver cuánto influye el hardware en este tipo de trabajos.

Otra cosa que costó fue entender las particularidades de Breakout y de los wrappers. Al principio no era obvio que el juego necesita la acción FIRE para lanzar la bola, y sin el wrapper que la presiona automáticamente el agente se podía quedar quieto y no aprender nada. También hubo que entender por qué el entorno se crea con frameskip=1 si al final se saltan 4 frames: el entorno de la versión 5 ya trae su propio salto de frames y AtariPreprocessing no deja aplicar el suyo encima, así que hay que desactivar uno de los dos.

10. Cómo ejecutar

Todos los comandos se ejecutan desde la carpeta `rl_games`. Primero se instalan las dependencias con `uv sync`. Si el computador tiene una tarjeta gráfica NVIDIA, se puede instalar PyTorch con CUDA dentro del mismo entorno y usar `--no-sync` en los comandos siguientes:

```
uv sync
uv pip install --reinstall-package torch "torch==2.10.0" --index-url
https://download.pytorch.org/whl/cu128
uv run --no-sync rlgames inspect --steps 5
uv run --no-sync rlgames train dqn --episodes 1000
uv run --no-sync rlgames load dqn --eval
uv run --no-sync rlgames render dqn --episodes 1
```

Si no hay GPU basta con quitar la segunda línea y el `--no-sync`.

Referencias

Farama Foundation. Arcade Learning Environment, Breakout. https://ale.farama.org/environments/breakout/

Farama Foundation. Gymnasium, Wrappers. https://gymnasium.farama.org/api/wrappers/

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278-2324. https://doi.org/10.1109/5.726791

Mnih, V., Kavukcuoglu, K., Silver, D. et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529 a 533.

Muñoz, E. (2026). *rl_games_sb* [Repositorio de GitHub]. https://github.com/emiliomunozai/rl_games_sb